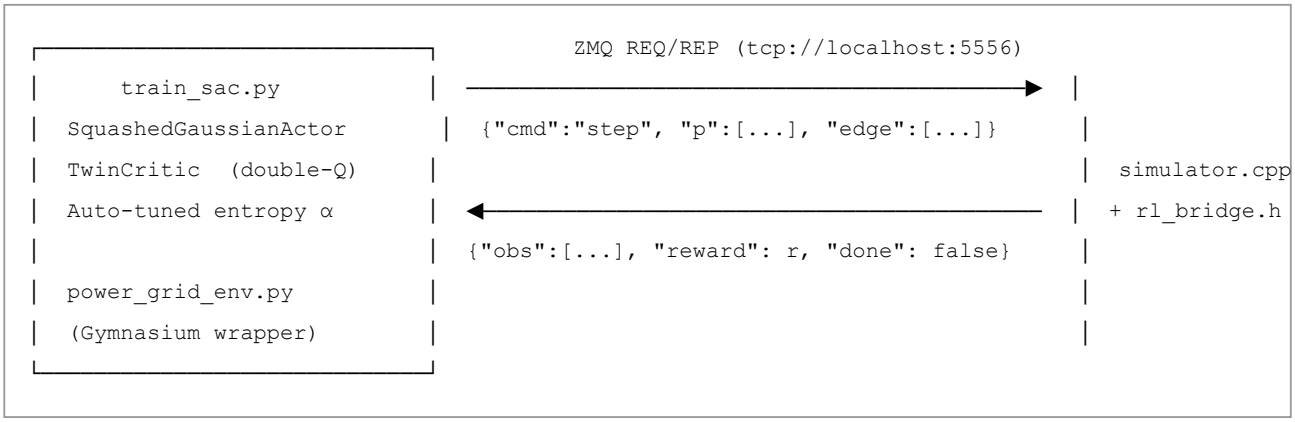
PowerGridRL — SAC-based Power Grid Control

A reinforcement learning system that trains a **Soft Actor-Critic (SAC)** agent to manage a simulated power grid in real time. A C++ simulator (with Raylib visualisation) runs the physics and communicates with a Python SAC agent over ZeroMQ. The agent controls **demand scaling per substation** and **transmission line topology** across a full simulated year (8 760 hourly steps per episode).

```
sac_implementation/
├── build/                      # CMake build output
├── src/
│   ├── cpp/
│   │   ├── simulator.cpp       # Power grid physics + Raylib visualisation
│   │   ├── rl_bridge.h         # ZMQ REP server + reward / obs computation
│   │   └── demand_data.csv     # Hourly demand schedule (8 760 rows × n_subs cols)
│   ├── python/
│   │   ├── power_grid_env.py   # Gymnasium wrapper (ZMQ REQ client)
│   │   └── train_sac.py        # SAC agent + training loop
│   └── CMakeLists.txt
```

How It Works



Action space `[ p_factors (n_subs) | edge_scores (n_edges) ]` ∈ `[0, 1]`

- `p_factors[i]` — demand scaling for substation *i* (1.0 = serve full demand, 0.0 = full shed)
- `edge_scores[i]` — > 0.5 connects the transmission line, ≤ 0.5 disconnects it

Observation space `[ edge_utils (n_edges) | sub_demands (n_subs) | time_feats (8) ]`

- `edge_utils[i]` — signed `current_load / max_load` (negative = reversed flow)
- `sub_demands[i]` — `current_demand / max_limit`
- `time_feats` — 8 cyclic sin/cos features encoding hour-of-day, day-of-week, month, week-of-year

Reward is computed entirely in C++ (`rl_bridge.h`) from 7 shaped components:

Component	Range	Purpose
`r_serve`	[0, 4]	Maximise fraction of demand served
`r_overload`	(−∞, 0]	Flat + quadratic penalty for tripped lines
`r_danger`	(−∞, 0]	Smooth early-warning ramp at util > 0.85
`r_cascade`	[−6, 0]	Superlinear penalty as trip fraction grows
`r_health`	[0, 0.3]	Bonus for lines in the healthy utilisation band
`r_shed`	[−1.5, 0]	Penalise curtailment only when grid is safe
`r_topology`	[−0.5, 0]	Discourage unnecessary disconnections

Total reward is clamped to **[−10, 10]** before being sent to Python.

Prerequisites

System

Requirement	Version
Linux (Ubuntu 20.04+ recommended)	—
GCC / G++	≥ 11
CMake	≥ 3.16
Python	≥ 3.9
CUDA Toolkit *(optional, for GPU training)*	≥ 11.8

C++ Libraries

Library	Purpose	Install
Raylib	Real-time grid visualisation	See below
ZeroMQ (libzmq)	C++↔Python IPC	`sudo apt install libzmq3-dev`
cppzmq	C++ header wrapper for ZMQ	`sudo apt install libcppzmq-dev`
nlohmann/json	JSON serialisation in the bridge	`sudo apt install nlohmann-json3-dev`
OpenGL / X11 / pthread	Raylib runtime deps (usually pre-installed)	`sudo apt install libgl1-mesa-dev libx11-dev`

Python Packages

Package	Purpose
`torch`	SAC neural networks (actor, twin-critic)
`gymnasium`	Standard RL environment interface
`pyzmq`	Python ZMQ client
`numpy`	Array math

Installation

1 — Clone the repository

```
git clone https://github.com/<your-username>/sac_implementation.git
cd sac_implementation
```

2 — Install system dependencies

```
sudo apt update
sudo apt install -y \
    build-essential cmake \
    libzmq3-dev libcppzmq-dev \
    nlohmann-json3-dev \
    libgl1-mesa-dev libx11-dev \
    libxrandr-dev libxinerama-dev libxcursor-dev libxi-dev   # Raylib X11 deps
```

3 — Install Raylib from source

Raylib 5.x is recommended. Ubuntu's package manager often ships an older version, so build from source:

```
git clone --depth 1 --branch 5.0 https://github.com/raysan5/raylib.git /tmp/raylib
cd /tmp/raylib
mkdir build && cd build
cmake .. -DBUILD_SHARED_LIBS=ON -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
sudo make install
sudo ldconfig
```

4 — Build the C++ simulator

```
cd sac_implementation
mkdir -p build && cd build
cmake ../src -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
```

The binary `powergrid_rl` and `demand_data.csv` will be placed in `build/`.

5 — Set up the Python environment

```
cd sac_implementation
python -m venv .venv
source .venv/bin/activate

pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118   # GPU
# OR for CPU-only:
# pip install torch torchvision

pip install gymnasium pyzmq numpy
```

> ***Note:*** *If you are using a different CUDA version, adjust the index URL. See pytorch.org/get-started (https://pytorch.org/get-started/locally/) for the right wheel URL.*

Running

The C++ simulator and the Python trainer are **two separate processes** that must run simultaneously. Start the simulator first so the ZMQ REP socket is bound before Python tries to connect.

Step 1 — Start the C++ simulator

Open a terminal and run:

```
cd sac_implementation/build
./powergrid_rl
```

You should see the Raylib window open and the following console output:

```
[Bridge] Running on tcp://*:5556
```

The simulator waits for commands from the Python agent. It will not advance until Python sends a `reset` command.

Step 2 — Start training (new session)

Open a second terminal:

```
cd sac_implementation
source .venv/bin/activate
cd src/python

python train_sac.py
```

Default: **200 episodes**, `n_subs=100`. The first ~3 episodes (~26 280 steps) are a **warm-up phase** where actions are sampled uniformly at random to pre-populate the replay buffer before gradient updates begin.

Step 3 — Resume training from a checkpoint

```
python train_sac.py --resume checkpoints_sac/best_sac.pt --episodes 300
```

Step 4 — Evaluate a saved policy (no gradient updates)

```
python train_sac.py --eval --resume checkpoints_sac/best_sac.pt
```

CLI Reference

```
usage: train_sac.py [-h] [--episodes N] [--resume PATH] [--eval]
                    [--n_subs N] [--hidden_dim N]
                    [--actor_lr F] [--critic_lr F]
                    [--batch N] [--replay N]
                    [--warmup N] [--reward_scale F] [--entropy_scale F]

Options:
  --episodes       Number of training episodes           (default: 200)
  --resume         Path to a .pt checkpoint to load      (default: None)
  --eval           Run in evaluation mode (no training)  (default: False)
  --n_subs         Substation count — MUST match C++     (default: 100)
  --hidden_dim     Hidden layer width for actor/critic   (default: 256)
  --actor_lr       Actor learning rate                   (default: 0)
  --critic_lr      Critic learning rate                  (default: 3e-4)
  --batch          Replay batch size                     (default: 256)
  --replay         Replay buffer capacity                (default: 200000)
  --warmup         Random-action warm-up steps           (default: 26280)
  --reward_scale   Scale applied to C++ rewards          (default: 0.05)
  --entropy_scale  Target entropy = -action_dim × this   (default: 0.005)
```

Outputs

Path	Contents
`src/python/checkpoints_sac/best_sac.pt`	Best checkpoint (by episode reward)
`src/python/checkpoints_sac/ep<N>_sac.pt`	Periodic checkpoint every 25 episodes
`src/python/logs_sac/run_<timestamp>.csv`	Per-episode metrics CSV
`build/rl_bridge_log.csv`	Per-step C++ debug log (reward components, util stats)

Metrics CSV columns

```
episode, total_reward, steps, overloads_total, max_severity,
avg_util_mean, power_shed_est, alpha,
critic_loss_mean, actor_loss_mean, total_steps
```

Key Hyperparameters

Parameter	Default	Notes
`hidden_dim`	256	Width of actor & critic MLPs
`critic_lr`	3e-4	Adam LR for twin critic
`tau`	0.005	Soft target-network update rate
`gamma`	0.99	Discount factor
`replay_capacity`	200 000	Circular buffer size
`min_replay` / `warmup_steps`	26 280	~3 episodes of random exploration before learning
`batch_size`	256	Mini-batch size for each gradient step
`reward_scale`	0.05	Scales C++ rewards (~100 range) down to ~1 for stable critic targets
`entropy_scale`	0.005	Target entropy = −`action_dim` × `entropy_scale`

Architecture Notes

SquashedGaussianActor — Two-tower encoder (grid features + time features), separate mean/log-std heads for `p_factors` and `edge_scores`. Output mapped to [0, 1] via `a = (tanh(u) + 1) / 2`.

TwinCritic — Two independent Q(s,a) networks; Bellman targets use the minimum of the two to reduce over-estimation bias. Target networks updated with exponential moving average (τ = 0.005).

Auto-tuned α — The entropy temperature is a learnable parameter updated to keep `E[-log π(a|s)] ≈ -action_dim × entropy_scale`. This removes the need for a manual exploration schedule.

Troubleshooting

`TimeoutError: C++ sim did not reply within 15000 ms` The simulator is not running or not yet ready. Make sure `./powergrid_rl` is started and has printed `[Bridge] Running on tcp://*:5556` before launching the Python script.

`ValueError: C++ obs has only N values, expected at least n_subs=100` The `--n_subs` argument passed to `train_sac.py` does not match the substation count compiled into the C++ simulator. Check the `n_subs` value in `simulator.cpp` and pass the matching value via `--n_subs`.

Raylib window does not open / `libraylib.so` not found Run `sudo ldconfig` after installing Raylib. If the library is installed to a non-standard prefix, add it to `LD_LIBRARY_PATH`:

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
```

CUDA out-of-memory during training Reduce `--batch` (e.g. `--batch 128`) or `--replay` (e.g. `--replay 100000`). Alternatively, training runs fine on CPU for smaller grids.

`demand_data.csv` not found The CMake post-build step copies `demand_data.csv` from `src/cpp/` into `build/` automatically. If you move the binary, copy the CSV alongside it.